

TP_PrograB

Guía integral para entender el programa

Diseño, datos, clases, métodos, simulación gradual, persistencia, PostgreSQL, JavaFX, reportes, pruebas y relación con la consigna del trabajo práctico.

Esta guía describe la versión del código presente en C:\Java\TP_PrograB al 25 de septiembre de 2026, incluidos cambios todavía no confirmados en Git. Distingue lo implementado, lo probado y lo pendiente de validación con PostgreSQL real.

Cómo leerla

- Primero seguí el recorrido de una partida nueva en los capítulos 1 a 3.
- Después estudié los objetos de dominio y la simulación en los capítulos 4 a 9.
- Finalmente recorré persistencia, base de datos, reportes, interfaz, pruebas y pendientes.

La idea central

Championship es el estado del torneo: contiene equipos, zonas, partidos, árbitros, países, ciudades, estadios, etapa y ganadores. Los controladores JavaFX reciben esa instancia y llaman a operaciones de la lógica. Los partidos simulados y sus incidencias se serializan en **campeonato.dat**.

Datos de origen	Objeto o servicio	Uso actual
torneo.json	TournamentLoader + DTO + TournamentData	Equipos, jugadores, DT, referís y países.
PostgreSQL Championship	CityRepository + StadiumRepository	Lectura y ABM; carga al crear y administración desde Venues.fxml.
campeonato.dat	ChampionshipRepository	Guardar y cargar la partida en curso por serialización.
FXML/CSS	View + Controller	Pantallas, botones, tablas, cuadro y reportes visibles.

Fuente principal: código fuente actual y consigna TPGrupal-2026-Campeonato Futbol (1).pdf, páginas 3 a 7 y 9 a 10.

1. Mapa del proyecto

Qué hay en cada carpeta y por qué se separó así.

Carpeta	Contenido	Responsabilidad
src/main/java/Core/Run	Championship, Main	Estado y coordinación de reglas del torneo; Main es entrada de consola para datos.
Core/loader + Core/dto	TournamentLoader, Validator, Data y DTO	Leer JSON, mapearlo a objetos reales y validar planteles.
Core/domain + Core/enums	Personas, equipos, partidos, tablas, alineaciones y enums	Representar las entidades y vocabulario del problema.
Core/simulation + Core/incidents	MatchSimulator y seis tipos de incidente	Crear resultados y registrar hechos del partido.
Core/persistence + Core/report + Core/console	Repositorio serializado, reportes y salida temporal	Guardar/cargar, calcular listados y mostrar diagnósticos.
Infrastructure/database	Conexión JDBC, repositorios y tests manuales	SELECT, INSERT, UPDATE y DELETE de ciudades/estadios.
GUI + Controller + resources	JavaFX, FXML, CSS, imágenes	Mostrar y operar el torneo mediante interfaz.
src/test/java	Tests de campeonato, etapas, desempates y sedes	Pruebas automatizadas sin alterar la partida real.

Dependencias, sin mezclar responsabilidades

La interfaz conoce a Championship y a los servicios de reporte/guardado. Championship conoce a las clases de dominio, al cargador y al simulador; no debería necesitar saber cómo se pintan botones. Los repositorios JDBC conocen Country, City y Stadium, no el algoritmo deportivo. Esa separación facilita probar reglas sin abrir JavaFX.

Código real - Mapa conceptual de dependencias; las flechas muestran quién entrega datos a quién

```
torneo.json --> DTO --> TournamentLoader --> Team/Player/Country
                                         |
                                         v
FXML --> Controller --> Championship --> MatchSimulator --> Match/Incident
                        |           |
                        |           +--> ChampionshipRepository --> campeonato.dat
                        +--> TournamentReportService --> tablas y listados
PostgreSQL --> CityRepository --> City --> StadiumRepository --> Stadium
                        |           |
                        +-----> VenuesController <-----+
```

PostgreSQL ya llega al flujo JavaFX: NewTournamentController intenta cargar sedes y VenuesController administra el ABM. Sin contraseña configurada, el torneo abre con una advertencia en lugar de bloquearse.

2. Cómo arranca la aplicación

Dos entradas distintas: JavaFX normal y herramientas de consola.

Entrada JavaFX

GUI/AppLauncher.java delega en **GUI/View.java**. View extiende Application y, en start(Stage), carga MainMenu.fxml, crea Scene, establece título y tamaño mínimo y muestra la ventana. FXMLLoader instancia MainMenuController porque el FXML declara fx:controller.

Código real - GUI/AppLauncher.java

```
public class AppLauncher {
    public static void main(String[] args) {
        View.main(args);
    }
}
```

Elegir Nuevo

MainMenuController carga NewTournament.fxml. Su controlador exige un nombre no vacío. Cuando el usuario confirma, crea **new Championship("torneo.json")**, intenta leer ciudades y estadios desde PostgreSQL, guarda el estado inicial y abre Tournament.fxml. El constructor programa 24 partidos de grupos pero no los juega.

Código real - Controller/NewTournamentController.java

```
Championship championship = new Championship(DATA_PATH);
loadVenues(championship);
new ChampionshipRepository().save(championship);
openTournament(championship, tournamentName);
```

Elegir Cargar

MainMenuController comprueba que exista campeonato.dat, llama repository.load() y entrega la instancia deserializada a TournamentController.setTournament(). Esta pantalla vuelve a calcular tabla y recupera la última fecha jugada. El nombre escrito al crear el torneo no está guardado en Championship: al cargar se muestra el literal **Loaded Tournament**.

Entradas auxiliares

- Core/Run/Main.java imprime una comprobación de los datos cargados desde JSON; no es el menú JavaFX.
- Core/Run/Championship.main ofrece un flujo temporal por terminal: fechas de grupos y después un Enter por encuentro eliminatorio.
- Infrastructure/database/*Test.java son pruebas manuales de JDBC, no pantallas del usuario final.

Crear una partida nueva sobrescribe campeonato.dat. Si querés conservar la partida que ya tenés, hacé una copia antes de pulsar Nuevo.

3. Del JSON a objetos Java

DTO representa la forma del archivo; dominio representa personas y equipos utilizables.

Qué significa DTO

Los DTO son estructuras sencillas con campos públicos y `@SerializedName`: Gson deposita allí el contenido de `torneo.json`. No simulan ni validan reglas deportivas. La jerarquía del archivo es torneo -> equipos/equipo -> plantel -> jugadores/jugador y dt; en paralelo están árbitros. Por ejemplo, `TeamDto.name` representa la clave JSON "nombre" y su campo `country` representa "pais".

Código real - `Core/dto/TeamDto.java`

```
@SerializedName("nombre") public String name;
@SerializedName("pais") public CountryDto country;
public int ranking;
@SerializedName("plantel") public SquadDto squad;
```

Secuencia del cargador

- `Files.readString` lee el archivo; `Gson.fromJson` crea `TournamentRootDto`.
- `mapTeams` recorre `TeamDto`, crea `Coach` y `Team`, numera personas y agrega cada `Player`.
- `mapReferees` crea `Referee` desde `RefereeDto`, con IDs a partir de 10.000.
- `countryFor` conserva un catálogo único por nombre normalizado; equipo, DT y referí pueden compartir la misma instancia `Country`.
- `TournamentData` agrupa teams, referees y countries; `TournamentValidator.requireValid` controla el tamaño del torneo y de cada plantel.

Cómo se interpreta una persona

`PersonDto` contiene tipo/número de documento, nombre completo y fecha. Las fechas usan `dd/MM/yyyy`. `mapPosition` traduce arquero/defensor/mediocampista/delantero a `Position`. Las características y estadísticas históricas del JSON se copian al `Player`, sin mezclarlas con los resultados nuevos del campeonato.

En este JSON, los nombres aparecen en orden apellido + nombre, por ejemplo "Mangalarga Juan". `splitFullName` toma la primera palabra como apellido y el resto como nombre. Ese supuesto funciona con la muestra, pero requiere atención para apellidos compuestos u otro formato de entrada.

Validación concreta

`TournamentValidator` exige 16 equipos; cada uno debe contener exactamente 18 jugadores: 2 arqueros, 6 defensores, 5 mediocampistas y 5 delanteros. También exige referís. `findDuplicateDocuments` existe para detectar documentos repetidos, pero `requireValid` no lo incorpora todavía como error automático.

Código real - `Core/loader/TournamentValidator.java`, extracto

```
if (data.getTeams().size() != 16) { ... }
if (team.getPlayers().size() != 18) { ... }
expected.put(Position.GOALKEEPER, 2);
expected.put(Position.DEFENDER, 6);
expected.put(Position.MIDFIELDER, 5);
expected.put(Position.FORWARD, 5);
```

4. Personas, países, ciudades y estadios

El modelo de objetos básico del torneo.

Herencia de personas

Person guarda ID, nombre, apellido, documento y nacimiento. Player, Coach y Referee heredan esos datos para no duplicarlos. Player añade camiseta, Position, suspensión, características y estadísticas históricas. Coach añade Country de nacionalidad y títulos. Referee añade Country de nacionalidad y años de referato.

Código real - Herencia real en Core/domain

```
Person
+-- Player (position, shirtNumber, isSuspended)
+-- Coach (nationality: Country, titlesWon)
+-- Referee (nationality: Country, refereeYears)
```

Country y el enlace con PostgreSQL

Country solo guarda name y define equals/hashCode por nombre sin distinguir mayúsculas. Team, Coach y Referee lo referencian. City tiene id long, name y un Country. Stadium tiene id long, name y una City. Así se puede navegar Stadium -> City -> Country, respetando la relación de estadio con ciudad y país con ciudad.

Código real - Relación entre clases; ilustración de lectura

```
Country country = city.getCountry();
City city = stadium.getCity();
String countryName = stadium.getCity().getCountry().getName();
```

Los países se crean al leer JSON; los repositorios de base de datos no crean otros Countries, sino que buscan el mismo objeto por nombre. Si PostgreSQL devuelve un país ausente del JSON, CityRepository lanza IllegalArgumentException. Es una decisión importante: el catálogo JSON es la referencia para países usados en este torneo.

Team

Team posee nombre, Country, ranking, Coach y hasta 18 Players. addPlayer impide null y más de 18 integrantes. getCompetitiveStrength combina ranking (35%), media de valoraciones (55%) y títulos del DT (10%). El ranking pequeño aporta más mediante 101 - ranking. Esta fuerza alimenta el simulador junto con ventaja local y un término sorpresa aleatorio.

Country, City y Stadium son clases de dominio diferentes. La base relacional conserva la ciudad y el estadio, mientras el objeto Country enlaza el dato cargado por JSON con las ciudades leídas por JDBC.

5. Zonas, calendario y tabla

Qué se programa al crear el campeonato y cómo avanza cada fecha.

Sorteo de cuatro zonas equilibradas

Championship ordena los 16 equipos por ranking, forma cuatro bombos de cuatro y mezcla cada bombo. En cada vuelta distribuye un equipo de ese bombo a cada zona A-D. Así ninguna zona obtiene los cuatro mejores o cuatro peores: queda un equipo por nivel de ranking.

24 partidos de grupos

Cada zona tiene cuatro equipos y seis enfrentamientos únicos. generateGroupMatches utiliza un schedule de tres jornadas, dos partidos por zona y fecha. En todo el campeonato son 4 zonas x 6 partidos = **24**; cada jornada tiene 4 zonas x 2 = **8** partidos. Los GroupMatch nacen pendientes (played=false) con fecha, local, visitante, referí y matchday 1, 2 o 3.

Código real - Core/Run/Championship.java - emparejamientos por jornada

```
int[][][] schedule = {
    {{0, 3}, {1, 2}},
    {{3, 2}, {0, 1}},
    {{1, 3}, {2, 0}}
};
```

Una fecha por pulsación

getCurrentMatchday toma el mínimo matchday de los GroupMatch aún no jugados. simulateNextMatchday filtra solo esos ocho pendientes, llama MatchSimulator.simulate a cada uno y no continúa por sí mismo a otra jornada. TournamentController.handleSimulateMatchday llama una sola vez al método, programa cuartos si termina la tercera, guarda y repinta tablas/resultados.

Código real - Controller/TournamentController.java

```
int playedMatchday = championship.getCurrentMatchday();
championship.simulateNextMatchday();
scheduleQuarterFinalsIfNeeded();
repository.save(championship);
showGroupStage();
```

Tabla de posiciones

TeamStanding acumula PJ, PG, PE, PP, GF, GC y puntos 3/1/0; DG se calcula GF - GC. getStandings no lee una tabla guardada: crea cuatro TeamStanding nuevos para la zona y vuelve a procesar sus partidos jugados. Eso mantiene la tabla derivada del historial real y permite recalcularla al cargar una partida.

Desempates completos

Championship.getStandings ya no contiene todo el algoritmo: crea un **StandingsCalculator** y le entrega la zona y los partidos. El calculador ordena primero por puntos, diferencia de gol y goles a favor. Cuando dos o más equipos siguen exactamente iguales, resolveTie construye una mini tabla usando solo sus enfrentamientos. Allí compara puntos, DG y GF; el ranking continental queda como último recurso determinista. getFirst y getSecond usan este mismo resultado, por lo que la clasificación a cuartos recibe el desempate corregido.

Código real: Championship.getStandings crea new StandingsCalculator() y retorna calculator.calculate(zone, matches). El algoritmo vive en Core/classification/StandingsCalculator.java.

6. Partidos e incidencias

La diferencia entre programación, resultado y hechos observables.

La clase base Match

Match es abstracta y guarda fecha, equipos, referí, Stadium opcional, marcador, lista de Incident, alineaciones iniciales y boolean played. Al crearse, el marcador es 0-0 y played=false. addIncident reordena por minuto y getIncidents devuelve una lista no modificable.

Cuatro clases de partido

Tipo	Datos particulares	Qué representa
GroupMatch	matchday; isGroupStage=true	Un encuentro único de zona; afecta la tabla.
FirstLegMatch	Sin marcador anterior	Ida de cuartos o semifinal; no define serie por sí sola.
SecondLegMatch	Marcador de ida y settledByPenalties	Vuelta y eventual definición de la serie.
FinalMatch	settledByPenalties	Partido único por el título.

Incident y polimorfismo

Incident es una clase abstracta con minute y getDescription(). Goal, YellowCard, RedCard, Substitution y PenaltyShootout son sus subtipos. Cada uno sobrescribe el acceso que el reporte necesita: Goal informa jugador goleador, arquero y si fue penal; las tarjetas informan afectado y puntos Fair Play; Substitution informa quién entra y sale; PenaltyShootout guarda pateador y conversión.

Código real - Polimorfismo usado por TournamentReportService

```
for (Incident incident : match.getIncidents()) {
    Player scorer = incident.getScoringPlayer();
    if (scorer != null) { /* sumar gol de partido */ }
}
```

Los tiros de una tanda son PenaltyShootout, no Goal. Por eso el ranking de goleadores cuenta goles durante el encuentro y no los de la definición. Goal de penal durante los 90 minutos sí cuenta; Goal en contra retorna null como goleador en el ranking.

Quién estaba realmente en cancha

Match.getPlayersOnField(team, minute) parte del once inicial y reproduce sustituciones y expulsiones hasta el minuto solicitado. Un cambio quita al que sale y agrega al que entra; una roja elimina al expulsado. MatchSimulator reutiliza esa consulta para tarjetas, cambios, goles y penales, evitando acciones posteriores de un jugador que ya no estaba habilitado.

Los hechos ya se almacenan cronológicamente. ChampionshipTest comprueba además que cada goleador estuviera en cancha en el minuto de su gol.

7. Alineaciones y simulación de 90 minutos

De una formación disponible a un partido con marcador y hechos.

FormationType y Lineup

FormationType enumera 4-3-3, 4-4-2, 3-5-2 y 4-5-1, junto con cantidades de defensores/mediocampistas/delanteros. Lineup.canUseFormation cuenta jugadores de cada posición no suspendidos y verifica que haya al menos un arquero y las cantidades requeridas. MatchSimulator elige aleatoriamente solo entre formaciones utilizables.

Lineup.selectBestLineup ordena jugadores disponibles por promedio de valoración descendente y agrega los necesarios por posición hasta formar once. Por eso la táctica se sortea, pero los titulares dentro de cada posición se eligen según valoración. Match.setInitialLineups exige exactamente once por equipo.

Código real - Core/simulation/MatchSimulator.java, extracto

```
List<FormationType> validFormation = Arrays.stream(FormationType.values())
    .filter(formation -> Lineup.canUseFormation(team, formation))
    .toList();
return validFormation.get(random.nextInt(validFormation.size()));
```

Fuerza, sorpresa y goles

El simulador calcula homeStrength = fuerza del equipo local + 3 y awayStrength = fuerza visitante. Añade surprise = random.nextGaussian() * 12. Luego estima goles para cada uno, limita la expectativa entre 0,25 y 3,5 y la transforma en un conteo aleatorio mediante productos de random.nextDouble(). Un equipo inferior puede ganar por el término sorpresa y el muestreo de goles.

Plan cronológico de hechos

Para cada equipo, MatchSimulator crea PlannedIncident con minuto y tipo YELLOW_CARD, RED_CARD o SUBSTITUTION. Ordena esos planes y los ejecuta consultando getPlayersOnField. Una sustitución busca un jugador activo y un suplente habilitado de la misma posición; una segunda amarilla genera roja; los goleadores se eligen entre quienes estaban presentes en su minuto.

Qué se registra

- Cada gol se agrega como Goal con minuto 1-90, autor, arquero rival, marca de penal y eventual marca de gol en contra.
- Tarjetas amarillas/rojas incluyen jugador y minuto; algunas rojas activan isSuspended para el siguiente encuentro.
- Hasta tres sustituciones por equipo se intentan con un suplente de la misma Position que el jugador que sale.
- Al finalizar se asigna played=true; antes de ese punto el partido no debe aparecer como resultado en la interfaz.

Cómo se cumple una suspensión

Antes de construir Lineup, simulate toma una fotografía de los suspendidos. Lineup los excluye del once y limpia la marca al considerar cumplida la fecha; la fotografía se conserva y también los excluye del banco. Así el jugador se pierde exactamente ese partido y queda disponible para el siguiente. ChampionshipTest verifica las tres condiciones.

Suspensiones y cronología están implementadas y probadas. Los minutos de participación de reportes todavía merecen casos extremos con varias incidencias en el mismo minuto.

8. Eliminatorias: un partido por clic

8 cuartos + 4 semifinales + 1 final = 13 encuentros.

Programar sin jugar

Al terminar fecha 3, stage cambia de GROUP_STAGE a QUARTER_FINALS.

TournamentController.scheduleQuarterFinalsIfNeeded calcula la última fecha de grupos + 7 días y llama Championship.scheduleQuarterFinals. Este método crea cuatro FirstLegMatch con los cruces 1A-2D, 1B-2C, 1C-2A y 1D-2B; no invoca al simulador. Si ya hay FirstLegMatch, no los duplica al cargar.

Orden real de las 13 pulsaciones

Pulsaciones	Qué ocurre	Estado luego del bloque
1-4	Se juegan las cuatro idas de cuartos, una por clic.	Se programan cuatro vueltas aún pendientes.
5-8	Se juegan las cuatro vueltas; al acabar se guardan cuatro ganadores.	stage=SEMI_FINALS; se programan dos idas.
9-10	Se juegan dos idas de semifinal.	Se programan dos vueltas.
11-12	Se juegan dos vueltas; se guardan dos finalistas.	stage=FINAL; se programa la final.
13	Se juega la final y eventual tanda de penales.	champion guardado; stage=FINISHED.

El método central

simulateNextKnockoutMatch verifica que grupos estén completos y que el torneo no esté terminado. Busca el primer Match no grupal con played=false, juega exclusivamente ese encuentro, llama advanceKnockoutStageIfNeeded para preparar lo que corresponda y devuelve el partido recién jugado. TournamentController lo guarda y reconstruye el cuadro. El test verifica que cada llamada suma exactamente uno al número de partidos jugados.

Código real: el método busca el primer partido no grupal con played=false, carga y asigna una sede disponible, llama MatchSimulator.simulate, confirma la baja de la sede y luego ejecuta advanceKnockoutStageIfNeeded.

Por qué se crean las vueltas después de las idas

SecondLegMatch almacena el resultado de la ida. Por eso no puede construirse correctamente antes de que esa ida sea jugada. Cuando terminan las cuatro idas, scheduleSecondLegs invierte local/visitante, añade siete días y copia los goles del primer encuentro. Se juega una vuelta por clic y recién después de todas se clasifican los ganadores.

Una sede distinta por encuentro

Cada Match eliminatorio posee Stadium. Al presionar Simular, Championship consulta nuevamente PostgreSQL, elige al azar uno de los estadios todavía disponibles, lo copia dentro del partido y recién entonces ejecuta MatchSimulator. Después intenta eliminar esa sede de la tabla estadio. Como la siguiente simulación vuelve a consultar la base, una sede eliminada ya no puede salir otra vez. Para completar las 13 eliminatorias sin reponer registros deben existir al menos 13 estadios.

La unicidad depende del ciclo PostgreSQL real: SELECT de disponibles, elección, simulación y DELETE. Esta edición no conserva el antiguo VenueAssignmentTest en memoria porque ya no representaba ese funcionamiento.

9. Cómo se decide un ganador

El reglamento del TP no es necesariamente el del fútbol actual.

Cuartos y semifinales: tres criterios

determineSeriesWinner compara primero los puntos de los dos partidos: victoria 3, empate 1, derrota 0. Si hay empate, compara la diferencia ponderada considerando los goles anotados de visitante con valor doble. Si sigue igual, secondLeg.setSettledByPenalties(true) y se ejecuta una tanda. La ida nunca ejecuta penales; la serie se resuelve tras la vuelta.

Código real - Representación fiel del orden de decisión

```
int pointsA = getSeriesPoints(firstLeg, secondLeg, teamA);
int pointsB = getSeriesPoints(firstLeg, secondLeg, teamB);
if (pointsA != pointsB) return pointsA > pointsB ? teamA : teamB;
int dgA = getWeightedGoalDifference(firstLeg, secondLeg, teamA);
int dgB = getWeightedGoalDifference(firstLeg, secondLeg, teamB);
if (dgA != dgB) return dgA > dgB ? teamA : teamB;
return determinePenaltyShootoutWinner(secondLeg);
```

Un ejemplo numérico

Si A gana la ida 1-0 y B gana la vuelta 2-1, ambos suman tres puntos. Los goles globales son A=2, B=2, pero el gol de A como visitante en la vuelta se pondera doble. La decisión usa diferencia ponderada, no solo el marcador global mostrado como dato informativo.

Final

FinalMatch es único. Si su marcador tras 90 minutos no empata, gana el de más goles. Si empata, se marca settledByPenalties y se decide por tanda. Se guardan cada pateador y si convirtió o falló como PenaltyShootout.

Tanda y lectura posterior

La tanda sortea cada remate con probabilidad 0,75: cinco intentos iniciales por equipo y luego muerte súbita. Los pateadores salen de match.getPlayersOnField(team, 90), por lo que incluyen cambios válidos y excluyen expulsados. getRecordedSeriesWinner y getRecordedFinalWinner leen el resultado guardado sin volver a ejecutar la tanda.

Todavía falta mostrar de forma explícita el criterio ganador (puntos, DG ponderada o penales) en todas las vistas, aunque el cálculo esté implementado.

10. Guardar, cargar y continuar

campeonato.dat conserva el estado de la partida; PostgreSQL cumple otra función.

Serialización

ChampionshipRepository.save usa ObjectOutputStream sobre campeonato.dat. Esto serializa Championship y el grafo de objetos alcanzables: Team, Player, Match, Lineup, Incident, Country, City, Stadium, listas, stage, quarterFinalWinners, finalists y champion. La operación sobrescribe el archivo anterior; la base PostgreSQL no reemplaza este guardado.

Código real - Core/persistence/ChampionshipRepository.java

```
try (ObjectOutputStream oos =  
    new ObjectOutputStream(new FileOutputStream(FILE_PATH))) {  
    oos.writeObject(championship);  
}
```

Qué se hace al recuperar

load lee ObjectInputStream. Durante la deserialización, Championship.readResolve reconstruye random y MatchSimulator porque son transient; repara jornadas antiguas sin matchday y deduce stage para archivos que no tenían ese atributo. También inicializa listas de ganadores ausentes en guardados anteriores. La etapa observada por partidos jugados puede elevar una stage vieja para impedir re-jugar fases terminadas.

En qué momentos se guarda

- Nuevo: inmediatamente tras construir el torneo, con 24 partidos pendientes.
- Grupos: después de cada fecha, incluidos sus ocho resultados y el eventual programa de cuartos.
- Eliminatorias: después de cada encuentro y de programar la ronda siguiente si correspondió.
- Al volver al menú o cerrar la ventana del torneo: TournamentController solicita otro guardado.

Prueba de continuidad

StagedSimulationTest conserva las comprobaciones de $24 + 13 = 37$ partidos, campeón persistente y rechazo de un avance tras FINISHED. Sin embargo, el flujo eliminatorio actual consulta PostgreSQL antes de cada encuentro; por eso esta prueba dejó de ser completamente aislada y necesita driver, contraseña y datos de sedes para recorrer el torneo completo. ChampionshipTest y StandingsTieBreakerTest sí pasaron en esta revisión sin escribir en la base.

Limitación: la semilla/posición interna del Random no se persiste. Se conservan resultados ya jugados, pero el azar para partidos futuros se recrea al cargar. ChampionshipRepository atrapa errores de lectura/escritura y los imprime, pero no los propaga al controlador.

11. Base de datos PostgreSQL y ABM

Lectura, escritura, pantalla administrativa y uso deportivo de las sedes.

JDBC, paso a paso

DatabaseConnection.connect carga org.postgresql.Driver, lee CHAMPIONSHIP_DB_PASSWORD del entorno y abre jdbc:postgresql://localhost:5432/Championship con usuario postgres. La contraseña no está escrita en el fuente. Si falta la variable o el driver, se informa un error.

Código real - Infrastructure/database/DatabaseConnection.java, extracto

```
String password = System.getenv("CHAMPIONSHIP_DB_PASSWORD");
return DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/Championship",
    "postgres", password);
```

Lectura relacional

CityRepository.findAll ejecuta SELECT de ciudad JOIN pais, busca el Country del catálogo JSON por nombre y crea City(id, name, country). StadiumRepository.findAll ejecuta SELECT de estadio JOIN ciudad JOIN pais, reutiliza la City ya cargada por ID y crea Stadium(id, name, city). Los try-with-resources cierran Connection, PreparedStatement y ResultSet.

Alta, modificación y baja

Repositorio	Operación	SQL y validación
CityRepository	create	INSERT ciudad(nombre,id_pais) RETURNING id; resuelve pais por nombre.
CityRepository	update	UPDATE nombre/id_pais por ID; exige nombre y Country.
CityRepository	delete	Cuenta estadios; rechaza la baja si existe alguno asociado.
StadiumRepository	create	INSERT estadio(nombre,ciudad) RETURNING id.
StadiumRepository	update	UPDATE nombre/ciudad por ID.
StadiumRepository	delete	DELETE por ID; la pantalla bloquea sedes asignadas a partidos.

Todos los valores de usuario se envían mediante PreparedStatement, no concatenando texto SQL. CityRepository consulta pais.id porque Country no guarda el ID relacional. Si el país del JSON no existe en la tabla pais, la operación se rechaza con un mensaje claro.

Venues.fxml y VenuesController

El botón Cities/Stadiums CRUD carga Venues.fxml. La pantalla presenta dos tarjetas con tabla y formulario: ciudad con país y estadio con ciudad. Muestra contadores de registros, filas seleccionadas con contraste y mensajes cuando una tabla está vacía. Update y Delete permanecen deshabilitados hasta seleccionar una fila; Add, Update y Delete llaman al repositorio correspondiente y las bajas piden confirmación. Volver reconstruye Tournament.fxml con la misma instancia Championship y la guarda.

Carga automática y tolerancia a fallos

NewTournamentController.loadVenues consulta primero ciudades y luego estadios, llama championship.loadVenues y continúa con el guardado. Si PostgreSQL o la contraseña fallan, muestra una advertencia pero deja crear el torneo. Dentro del ABM, cada modificación vuelve a leer ambas listas, actualiza Championship y guarda campeonato.dat.

Uso deportivo

Match incorpora Stadium y no permite cambiarlo una vez jugado. Antes de cada eliminatoria, reloadVenuesFromDatabase ejecuta CityRepository.findAll y StadiumRepository.findAll. Se elige una sede al azar, se simula el encuentro y completePendingKnockoutMatch ejecuta DELETE. Si la baja falla, pendingKnockoutMatch conserva el partido ya jugado: el siguiente clic reintenta solamente la baja y no genera otro resultado. showMatchDetails presenta estadio y ciudad junto con fecha, formaciones e incidencias.

Validación pendiente: el código compila y el FXML es válido, pero CHAMPIONSHIP_DB_PASSWORD no estaba disponible en esta terminal. El ciclo real SELECT/INSERT/UPDATE/DELETE y las 13 bajas automáticas deben probarse contra Championship.

12. Reportes y estadísticas

Los valores se calculan de partidos jugados e incidencias; no son los históricos del JSON.

TournamentReportService

Es un servicio de consulta: recibe Championship, recorre getPlayedMatches y construye filas para distintos listados. Los controladores convierten las filas a texto en reportArea. Al consultar en cualquier momento, los reportes reflejan solo lo disputado hasta entonces.

Método	Fuente del cálculo	Vista actual
getTopScorers	Goal de partido, normal o de penal; excluye gol en contra y tanda	Jugador, equipo, goles y goles de penal.
getParticipation	Once inicial, entradas, salidas y expulsiones	Ranking de partidos/minutos estimados.
getFairPlay	YellowCard=1; RedCard=3 por jugador/equipo	Equipo, tarjetas y puntos; menos puntos primero.
getTeamStats	Edad, DT y TeamStanding de grupos	Edad media, DT, GF/GC, efectividad.
getPlayerStats	Participación, goles y goles recibidos por arquero	Jugador/posición, PJ, minutos y goles.
getRefereeStats	Partidos jugados dirigidos y años históricos	Referí, partidos y años.

Filas auxiliares

TopScorerRow, ParticipationRow, FairPlayRow, TeamStatsRow, PlayerStatsRow y RefereeStatsRow son clases internas de TournamentReportService. Representan un renglón de salida con getters; no se serializan como parte esencial del torneo ni son tablas de PostgreSQL.

Cuidado con la participación

getPlayedMinutes devuelve 90 para un titular salvo sustitución o roja; a un suplente le atribuye 90 - minuto de entrada. Las incidencias ahora llegan ordenadas y coherentes con el campo, pero conviene agregar pruebas específicas para expulsión de suplentes, cambios múltiples y hechos en el mismo minuto antes de declarar todos los promedios exactos. Los históricos de Player.statistics no se suman al campeonato.

Pendiente de la consigna: identificaciones PDF con foto/código de barras, filtro Todos/una Position, promedio de goles recibidos por arquero, promedio de años de referato al pie y exposición completa de edad/nacionalidad del DT en la vista. Get Identifications es aún un aviso, no un PDF.

13. Interfaz JavaFX de punta a punta

FXML diseña; Controller responde; Championship resuelve las reglas.

Los cuatro FXML

- MainMenu.fxml: Nuevo, Cargar, Stats y Exit. Stats del menú todavía imprime un mensaje pendiente.
- NewTournament.fxml: nombre y confirmación; el nombre se usa en la sesión, no queda persistido.
- Tournament.fxml: barra lateral de grupos, fechas, knockout, partido siguiente, rankings y reportes; tabla de posiciones, área de resultados, cuadro y reportArea.
- Venues.fxml: tablas y formularios para altas, modificaciones y bajas de ciudades y estadios.

style.css y las imágenes BackgroundImage.jpg/Trophy.gif son presentación. Cambiar colores o animaciones no cambia los puntos, clasificación o resultados. FXMLLoader une fx:id con campos @FXML y onAction con métodos del controlador.

Qué hace TournamentController

- setTournament conserva Championship, programa cuartos si corresponde a un guardado previo, muestra grupos y actualiza habilitación de botones.
- handleSimulateMatchday juega una jornada, guarda y pinta tabla/resultados de la última fecha.
- handleSimulateKnockoutMatch juega un único encuentro, guarda, reconstruye el cuadro y deshabilita el botón al terminar.
- handleKnockoutStage solo cambia de vista; no simula partidos al abrir el cuadro.
- buildKnockoutBracket arma tarjetas de series pendientes, parcialmente jugadas y resueltas; un clic en una tarjeta jugada abre formaciones e incidencias.
- handleTopScorers, handleFairPlay y demás handlers llaman al servicio de reportes y muestran texto.
- handleVenuesCrud carga Venues.fxml y entrega la misma Championship; VenuesController refresca PostgreSQL y vuelve sin perder el torneo.

Visibilidad y distribución

showGroupTable muestra tabla y matchdayResultsArea. showReport muestra reportArea. showKnockoutBracket muestra el cuadro. Cada uno usa visible y managed: ocultar sin setManaged(false) dejaría espacio vacío. showZone cambia la zona de la tabla sin re-simular partidos.

Código real - src/main/resources/Tournament.fxml, resumido

```
<Button fx:id="simulateMatchdayBtn"
        onAction="#handleSimulateMatchday"
        text="Simulate Next Matchday" />
<Button fx:id="simulateKnockoutBtn"
        onAction="#handleSimulateKnockoutMatch"
        text="Simulate Next Knockout Match" />
```

Estado visual actual: los resultados de grupo se listan por fecha y el cuadro permite ver incidencias eliminatorias. La consigna pide tabla antes y después de cada partido de grupos; esa comparación individual todavía no se presenta.

14. Catálogo: clases de entrada, carga y DTO

Cada renglón indica dónde está la clase y qué aporta al proyecto.

Clase	Carpeta	Papel exacto
AppLauncher	GUI	main que inicia View sin extender Application.
View	GUI	Application que carga MainMenu.fxml y muestra Stage/Scene.
MainMenuController	Controller	Navega Nuevo/Cargar; Exit; Stats del menú pendiente.
NewTournamentController	Controller	Valida nombre, construye Championship, guarda y abre torneo.
TournamentController	Controller	Botones de fechas/partidos, tablas, cuadro, incidentes, reportes y guardado.
VenuesController	Controller	ABM JDBC de ciudades/estadios y regreso al torneo activo.
Main	Core/Run	Chequeo básico del JSON por consola; no es la GUI.
Championship	Core/Run	Estado y coordinación deportiva de todas las etapas.
TournamentLoader	Core/loader	Gson, DTO -> Country/Team/Coach/Player/Referee.
TournamentData	Core/loader	Contenedor de equipos, referís y países ya convertidos.
TournamentValidator	Core/loader	Reglas de 16 equipos y plantilla 2/6/5/5; duplicados aparte.
TournamentRootDto	Core/dto	Nodo raíz torneo.
TournamentDto	Core/dto	Nodos equipos y árbitros.
TeamsDto	Core/dto	Lista equipo del JSON.
TeamDto	Core/dto	Nombre, país, ranking y plantel de un equipo.
SquadDto	Core/dto	Jugadores y DT del plantel.
PlayersDto	Core/dto	Lista jugador.
PlayerDto	Core/dto	Posición, persona, características y estadísticas históricas.
CoachDto	Core/dto	Datos personales, país y títulos del DT.
RefereesDto	Core/dto	Lista arbitro.
RefereeDto	Core/dto	Datos personales, país y años de referato.
PersonDto	Core/dto	Documento, nombre completo y fecha de nacimiento.
CountryDto	Core/dto	Nombre de país recibido como valor JSON simple.

Los DTO terminan al salir del cargador; la simulación trabaja con objetos de dominio. Esto evita que un @SerializedName o la forma concreta del JSON se filtren en MatchSimulator.

15. Catálogo: dominio y enums

Entidades, relaciones y tipos de partido.

Clase	Carpeta	Papel exacto
Person	Core/domain	Datos personales heredados por jugador, DT y referí.
Player	Core/domain	Posición, dorsal, suspensión, valoraciones e históricos.
Coach	Core/domain	DT: Country y títulos.
Referee	Core/domain	Referí: Country y años en el referato.
Country	Core/domain	País por nombre; igualdad insensible a mayúsculas.
City	Core/domain	Ciudad con ID long y Country; leída desde PostgreSQL.
Stadium	Core/domain	Estadio con ID long y City; sede asignable a Match eliminatorio.
Team	Core/domain	Equipo con plantel, DT, ranking y fuerza competitiva.
TournamentZone	Core/domain	Zona A-D con cuatro Teams; protege su lista.
TeamStanding	Core/domain	Tabla derivada de partidos grupales jugados.
Lineup	Core/domain	Formación y once titulares por equipo.
Match	Core/domain	Base abstracta de resultado, estadio, lineups e incidencias cronológicas.
GroupMatch	Core/domain	Partido de zona identificado por fecha/jornada.
FirstLegMatch	Core/domain	Ida de una serie eliminatoria.
SecondLegMatch	Core/domain	Vuelta con marcador anterior y marca de penales.
FinalMatch	Core/domain	Encuentro único por el campeonato.
Position	Core/enums	GOALKEEPER, DEFENDER, MIDFIELDER, FORWARD.
FormationType	Core/enums	Cuatro tácticas y sus cantidades por posición.
TournamentStage	Core/enums	GROUP_STAGE -> QUARTER_FINALS -> SEMI_FINALS -> FINAL -> FINISHED.

Qué concepto de POO muestra este grupo

Person/Player/Coach/Referee y Match/GroupMatch/FirstLegMatch/SecondLegMatch/FinalMatch ejemplifican herencia. Las listas expuestas como unmodifiableList y addPlayer con validaciones ayudan al encapsulamiento; sigue habiendo setters amplios que permiten estados inválidos y merecen revisión.

16. Catálogo: simulación, incidencias y servicios

Dónde se calcula cada efecto del torneo.

Clase	Carpeta	Papel exacto
MatchSimulator	Core/simulation	Formación, suspensión, plan cronológico, goles, tarjetas y cambios válidos.
Incident	Core/incidents	Base abstracta con minuto y consultas polimórficas.
Goal	Core/incidents	Gol de partido, autor, arquero, penal y gol en contra.
YellowCard	Core/incidents	Amonestación: jugador y 1 punto Fair Play.
RedCard	Core/incidents	Expulsión: jugador y 3 puntos Fair Play.
Substitution	Core/incidents	Minuto, jugador que entra y el que sale.
PenaltyShootout	Core/incidents	Remate de definición: pateador y convertido/fallado.
ChampionshipRepository	Core/persistence	Serializa y deserializa campeonato.dat.
TournamentReportService	Core/report	Calcula seis tipos de filas/rankings desde partidos jugados.
MatchConsoleReporter	Core/console	Imprime formación, eventos y marcador; apoyo temporal.
StandingsConsoleReporter	Core/console	Imprime tabla por zona en el flujo terminal.
DatabaseConnection	Infrastructure/database	Conexión JDBC a Championship; contraseña por entorno.
CityRepository	Infrastructure/database	SELECT/INSERT/UPDATE/DELETE y restricción de ciudad con estadios.
StadiumRepository	Infrastructure/database	SELECT/INSERT/UPDATE/DELETE; PostgreSQL genera el ID del estadio.
DatabaseTest	Infrastructure/database	Test manual para confirmar conexión PostgreSQL.
CityRepositoryTest	Infrastructure/database	Test manual de lectura de ciudades.
StadiumRepositoryTest	Infrastructure/database	Test manual City -> Stadium -> Championship.loadVenues.
StandingsTieBreakerTest	src/test/java	Empate total, enfrentamiento directo y ranking de respaldo.
StandingsCalculator	Core/classification	Calcula la tabla y resuelve desempates fuera de Championship.
Paths	Utils	Constante de ruta del FXML de menú; casi no interviene.

Las seis Row internas de TournamentReportService son objetos de presentación de resultados calculados, no entidades persistentes de PostgreSQL. Cada handler de TournamentController elige el reporte, transforma sus filas en líneas legibles y las coloca en reportArea.

17. Una partida concreta, paso a paso

Lectura en voz alta del estado para estudiar el flujo completo.

Momento	Championship.getMatches	getPlayedMatches	stage y efecto visible
Nuevo	24	0	GROUP_STAGE; zonas sorteadas, tabla sin resultados.
Primer clic Fecha	24	8	Se ve MATCHDAY 1 y tabla recalculada.
Segundo clic Fecha	24	16	Se ve MATCHDAY 2; ninguna eliminatoria jugada.
Tercer clic Fecha	28	24	QUARTER_FINALS; cuatro idas programadas, 0 eliminatorias jugadas.
Primer clic Knockout	28	25	Una ida jugada; se muestra ida/pendiente y sus incidencias.
Cuarto clic Knockout	32	28	Cuatro idas jugadas; cuatro vueltas programadas.
Octavo clic Knockout	34	32	Cuatro ganadores guardados; dos idas de semi programadas.
Duodécimo clic Knockout	37	36	Dos finalistas guardados; final pendiente.
Décimo tercer clic Knockout	37	37	FINISHED, champion persistente, botón deshabilitado.

Qué no se duplica al cargar

Los GroupMatch tienen played=true tras su fecha y getCurrentMatchday ignora los jugados. scheduleQuarterFinals detecta FirstLegMatch previos. simulateNextKnockoutMatch busca solo el primer no jugado y prohíbe avanzar desde FINISHED. El estado de ganadores y stage queda serializado. La prueba StagedSimulationTest hace varios cierres/cargas en memoria para comprobar ese comportamiento.

Qué ocurre con las sedes durante el recorrido

Al crear, NewTournamentController intenta cargar PostgreSQL. Los 24 partidos de zona no requieren Stadium. En cada clic eliminatorio se recarga la lista disponible, se elige una sede al azar y se elimina después de usarla. Por eso no se asignan anticipadamente las sedes de toda una ronda: el estado vigente de PostgreSQL decide la sede del partido actual.

Si algo no aparece

- Si Nuevo parece iniciar un torneo terminado, verificá que estés ejecutando este proyecto y no otra copia; NewTournamentController actual no llama simulateGroupStage.
- Si el botón de fecha está deshabilitado, comprobá hasPendingGroupMatchdays() y si el archivo cargado ya completó grupos.
- Si el botón eliminatorio está deshabilitado, revisá getStage(): GROUP_STAGE y FINISHED lo deshabilitan.
- Si una sede no aparece, comprobá CHAMPIONSHIP_DB_PASSWORD, la base Championship y que todavía existan estadios disponibles. Para completar el torneo sin volver a cargarlos se necesitan 13.

La diferencia entre getMatches (programados) y getPlayedMatches (resultados reales) es la clave para leer esta guía y para depurar la interfaz.

18. Relación con la consigna y pendientes

No confundas una clase existente con un requisito ya cumplido de punta a punta.

Requisito de la consigna	Estado en este código	Qué falta o verificar
16 equipos, zonas balanceadas, 24 grupos	Implementado	Pruebas adicionales de sorteo y entrada inválida.
Simulación no determinística, alineación, goles/incidencias	Implementado y probado	Revisión visual y casos extremos de minutos.
Avance por fechas y luego por encuentro	Implementado	Prueba manual JavaFX y experiencia de usuario.
Guardado para retomar estadio	Implementado y probado en memoria	Prueba real con campeonato.dat y errores de IO.
Desempate de grupos por enfrentamiento directo	Implementado y probado	Mostrar criterio aplicado en la interfaz.
Tabla antes/después de cada resultado	Incompleto	Mostrar comparación individual en JavaFX.
Ganador por puntos, DG visitante doble, penales	Implementado	Exponer criterio ganador en todas las vistas.
Estadios al azar sin repetir en 13 encuentros	Implementado en código	Recorrido real completo contra PostgreSQL.
ABM ciudad/estadio en PostgreSQL	Implementado en código	Prueba manual de escritura contra PostgreSQL real.
Rankings, listados y cuadro	Parcial	Exactitud minutos; filtros/promedios/campos faltantes.
Identificaciones PDF foto/barcode	Pendiente	Generación y datos/fotografías.
Documentación/ágil/inglés/presentación	Fuera del flujo funcional	Verificar evidencias y entregables con el grupo.

Siguiente corrección prioritaria

La base deportiva crítica ya incorpora suspensiones, cronología, pateadores y desempate directo. La prioridad pasa a validar el ABM con la base real, completar identificaciones PDF y cerrar los campos/filtros faltantes de reportes. También debe mostrarse el criterio de cada clasificación y la tabla antes/después de cada resultado individual.

Esta guía explica lo existente y señala lo pendiente. No equivale por sí sola a declarar aprobado el TP; la consigna exige funcionalidad completa, validaciones, interfaz amigable, trabajo grupal e inglés.

19. Dónde mirar y cómo estudiarlo

Ruta corta para volver del PDF al código sin perderse.

Pregunta	Abrió primero	Después miró
¿De dónde salen los equipos?	Core/loader/TournamentLoader.java	Core/dto y torneo.json.
¿Cómo se forman las zonas?	Core/Run/Championship.java: drawBalancedZones	TournamentZone y ranking de Team.
¿Por qué una fecha tiene ocho juegos?	Championship: generateGroupMatches	GroupMatch.getMatchday.
¿Cómo se obtiene un marcador?	Core/simulation/MatchSimulator.java	Team.getCompetitiveStrength y Lineup.
¿Dónde quedan los eventos?	Core/domain/Match.java	Core/incidents/* y showMatchDetails.
¿Quién clasifica?	Championship: getStandings, determineSeriesWinner	TeamStanding y reglas de la consigna.
¿Cómo avanza un partido?	Championship: simulateNextKnockoutMatch	advanceKnockoutStageIfNeeded y TournamentController.
¿Por qué no se repite al cargar?	ChampionshipRepository + readResolve	stage, played y StagedSimulationTest.
¿Qué viene de PostgreSQL?	Infrastructure/database/*Repository	City, Stadium y loadVenues.
¿Dónde está el ABM?	Controller/VenuesController.java	Venues.fxml y métodos create/update/delete.
¿Cómo se asigna una sede?	Championship: simulateNextKnockoutMatch	reloadVenuesFromDatabase, Match.stadium y DELETE posterior.
¿Cómo funciona un ranking?	Core/report/TournamentReportService.java	Incident polimórfico y handlers de reportArea.

Preguntas para defender el diseño

- Explicá por qué DTO no es Team: el primero refleja la forma del JSON; el segundo participa de reglas del torneo.
- Explicá por qué TeamStanding se recalcula desde resultados jugados y no desde los históricos de Player.statistics.
- Explicá la diferencia entre crear un partido pendiente y simularlo; played es el indicador del resultado real.
- Explicá por qué una vuelta nace después de las idas y por qué una serie puede requerir una tanda.
- Explicá por qué el ABM usa PreparedStatement y por qué no puede borrarse una ciudad con estadios.
- Explicá por qué borrar de PostgreSQL la sede usada impide que vuelva a aparecer en el SELECT del partido siguiente.
- Mostrá herencia, polimorfismo y encapsulamiento con ejemplos concretos de Person, Match e Incident.

Fuentes y límites de esta edición

Código fuente local en C:\Java\TP_PrograB\src\main\java y src\main\resources; pruebas en src\test\java. Consigna local TPGrupal-2026-Campeonato Futbol (1).pdf, requisitos de páginas 3-7 y 9-10. Se describe el código integrado al 25/09/2026. La compilación, el FXML, ChampionshipTest y StandingsTieBreakerTest pasaron; no se ejecutaron escrituras ni el recorrido eliminatorio completo porque CHAMPIONSHIP_DB_PASSWORD no estaba configurada. Identificaciones PDF sigue pendiente.